



MASTER IN
COMPUTER
SCIENCE

Evaluating Processing in-Memory Hardware to Accelerate Homomorphic Encryption

Master Thesis

Joel Hari

Philosophisch-naturwissenschaftlichen Fakultät
der Universität Bern

June 2024

u^b

UNIVERSITÄT
BERN

unine
UNIVERSITÉ DE
NEUCHÂTEL

**UNI
FR**

UNIVERSITÉ DE FRIBOURG
UNIVERSITÄT FREIBURG

Abstract

Homomorphic encryption schemes allow computations to be performed on encrypted data. Homomorphic encryption is an excellent tool to eliminate security risks while executing an application on an external server. However, due to computational cost and the memory-intensive nature of homomorphic encryption algorithms, current homomorphic encryption schemes are infeasible to use. Hence, we search for a way to accelerate homomorphic encryption operations.

Modern CPU-centric computer systems face a substantial memory bottleneck when executing memory-intensive applications. The processing in-memory paradigm essentially eliminates this bottleneck by equipping the memory subsystem with computing capabilities. Compute units placed near or in the memory have direct high-bandwidth and low-latency access to stored data. The processing in-memory architecture has key performance and energy cost advantages over a CPU-centric system.

We aim to evaluate the potential of using processing in-memory to accelerate homomorphic encryption. We focus on accelerating the popular homomorphic encryption library HELib. We implement different methods from the library on processing in-memory hardware and compare the performance to the CPU. We gained important insights into how processing in-memory can be used to accelerate computations within HELib and the limitations of processing in-memory in the same context.

Keywords - processing in-memory, homomorphic encryption, HELib

Prof. Dr. Pascal Felber, Complex Systems, Department of Computer Science, University of Neuchâtel, Supervisor

Dr. Peterson Yuhala, Complex Systems, Department of Computer Science, University of Neuchâtel, Co-Supervisor

Contents

1	Introduction	1
2	Background	2
2.1	Homomorphic Encryption	2
2.1.1	HElib	2
2.2	Processing in-Memory	3
2.2.1	UPMEM PIM-DRAM	3
3	Related Work	4
3.1	Homomorphic Encryption	4
3.1.1	Accelerating HE operations using GPUs	4
3.1.2	Accelerating HE operations using FPGAs	4
3.2	Processing in-Memory	5
3.2.1	UPMEM PIM-DRAM	5
3.2.2	Homomorphic Encryption on UPMEM PIM-DRAM	5
3.3	Trusted Execution Environment	5
4	Methodology	6
4.1	Most Called Methods	7
4.2	Most Cache Misses	8
4.3	Polynomial Addition	9
5	Evaluation	10
5.1	FFT Butterfly Loop	10
5.2	New FFT Layer	12
5.3	COBRA	13
5.4	Polynomial Addition	14
6	Discussion	16
6.1	Limitations	17
7	Conclusion	19
8	Future Work	20

1

Introduction

Cloud computing enables network access to a shared pool of resources (*e.g.*, networks, servers, storage) [1]. These cloud-based computing methods are valued for their scalability, cost-efficiency, and flexibility, making them ideal for a wide range of applications.

With conventional encryption schemes, the data that is transferred from and to the server and the data stored on the server are all encrypted. However, when we need to perform any operations on that data (*e.g.*, search a database), the data needs to be decrypted to perform the operation. This means the data must be in plaintext on the server at some point. The hardware and software used to process our data are under complete control of the server operator, and thus, the latter can access the sensitive data if decrypted. Hence, we will always have a security risk as we cannot trust the server operator. Further, we need to provide the server with the key for decryption. This is another security risk, as a third party might gain access to that key. These security risks motivate the need for tools like homomorphic encryption (HE).

HE schemes allow operations to be performed directly on the encrypted data [2, 3]. This means that the data will never be decrypted on the server. Therefore, even a malicious server cannot read the cleartext data, and thus, we can guarantee the security of our application. However, HE schemes come with a high computational overhead and are highly memory-intensive [4, 5].

The memory bandwidth forms a significant bottleneck in modern CPU-centric systems that hinders the performance of memory-intensive applications [5–7]. Processing in-memory (PIM) tries to overcome this memory bandwidth bottleneck by enabling the memory subsystem to perform computations. Compute units placed in the memory have direct high-bandwidth and low-latency memory access, essentially eliminating the memory bandwidth bottleneck, increasing the performance of memory-intensive applications [6].

This study aims to evaluate if we can accelerate parts of the HE library HELib using PIM. We implement different methods from HELib on the UPMEM DRAM processing unit (DPU) processor and compare their performance to the performance of the original HELib implementations. In the first step, we evaluate which parts/methods of the HELib library are best suited for an implementation using the UPMEM DPUs. We look at previous work to understand what kind of workloads are best suited for the DPUs. We then use different profiling approaches to identify HELib methods that are performing such workloads. We evaluate our work by comparing execution times from our implementations to the execution times of the original HELib implementations. This gives us insight into what parts of HELib can or cannot be accelerated using the UPMEM DPU processor.

2

Background

In this chapter, we introduce the relevant background on homomorphic encryption and processing in-memory.

2.1 Homomorphic Encryption

A fully homomorphic encryption (FHE) scheme allows one to perform ciphertext computations without ever needing to decrypt the data at any point. FHE was first introduced by Gentry [2, 3]. He developed an encryption scheme allowing to perform both additions and multiplications over encrypted data. These operations correspond to the boolean AND and XOR. Since any computation can be rewritten as the result of only AND and XOR operations, any application can be implemented using a FHE scheme. However, this generates a significant overhead since even simple computations can take a long time. Performing any operation takes significantly longer than it takes to perform the same operation on the decrypted data [4]. This is also due to the memory-intensive nature of the HE algorithms. The result is that FHE applications are costly on computational resources and runtime.

As of now, there is no FHE scheme that is practical to use because of the computational cost [4]. Nevertheless, several HE libraries are available, each with its own strengths and weaknesses [8]. For our project, since DPU programs have to be written in C, we would prefer to use a library written in C. However, no HE library written in C that is well-maintained and up-to-date is available.

2.1.1 HELib

We chose to work with the HELib library [9–12] for our project. HELib is an open-source, fully homomorphic encryption library distributed under the terms of the Apache License v2.0. Access to the complete source code is essential since we want to integrate methods implemented for the DPUs directly into the source code. HELib is written in C++, which means we have to rewrite any method we want to implement for the DPUs in C.

2.2 Processing in-Memory

Processor-centric computer architectures have a central processing unit (CPU) connected to the memory via a bus. Whenever the CPU needs data that is not in one of the on-chip caches (*e.g.*, causes a cache miss), the data must be loaded from memory. Loading data from memory usually takes 100 to 300 CPU cycles [13]. Thus, with data-intensive applications, the CPU might spend many cycles waiting for data to be transferred. Further, CPU performance has seen a significant increase over the years. Meanwhile, the memory capacity and memory bandwidth did not increase as much [6]. Thus, the memory bandwidth forms a significant bottleneck in modern CPU-centric systems that hinders the performance of data-intensive applications.

PIM tries to overcome this memory bandwidth bottleneck by enabling the memory subsystem to perform computations [6]. Compared to CPU-centric systems, the PIM architecture has key performance and energy cost advantages. This is mainly due to the minimization of data movement in a PIM system [6]. Generally, there are two approaches to implementing the PIM architecture: processing using memory (PUM) and processing near memory (PNM). PUM describes techniques that allow to perform computations in the memory without any special compute units. PUM can be achieved by, at most, minimally modifying the existing DRAM architecture. Making use of the physical properties of DRAM, one can for example perform bitwise operations between DRAM rows [6, 14]. PNM involves integrating compute units inside or close to the memory banks. These compute units have direct high-bandwidth and low-latency access to data stored in memory.

2.2.1 UPMEM PIM-DRAM

The company UPMEM has developed the UPMEM PIM-DRAM module, the first commercially available PIM hardware. UPMEM PIM-DRAM is a DRAM module equipped with 128 DRAM processing units (DPUs), each of which is assigned to its own 64 MB memory bank. The DPU is a general-purpose processing unit that implements a RISC-based instruction set architecture (ISA). The ISA supports native 32-bit integer additions and subtractions and 8-bit integer multiplication. All other more complex operations are software emulated [15, 16]. The PIM-DRAM module can be installed on the motherboard in any conventional DDR4 DIM slot. Since each memory bank has two operating modes, making it either belong to the CPU or the DPU, the CPU can use the PIM-DRAM like any other DRAM module. The UPMEM PIM architecture provides extensive parallelism, which can be leveraged to provide better performance than the CPU for highly parallelizable applications.

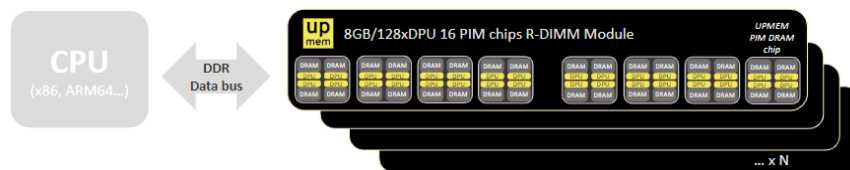


Figure 2.1: Schematic diagram of a UPMEM PIM DRAM module [16].

The DPUs operate independently of each other on their own memory bank. The communication and synchronization of the DPUs has to be done via the CPU.

To create, run, and debug DPU programs, UPMEM provides its UPMEM DPU toolchain [17]. For the development of DPU programs, the toolchain includes a C compiler to build the DPU programs, and a DPU Runtime Library that provides a set of DPU-specific functions and a subset of the standard C library. Hence, DPU programs have to be written in C. Further, the toolchain provides a host library for developing host applications that run on the CPU. Essentially, the host library provides functions to read and write into the DPU memories and start the DPU programs.

3

Related Work

In this chapter, we cover a set of related work. We look at different approaches to accelerate HE operations instead of using PIM. We look at research on the UPMEM PIM architecture to understand which workloads are best suited for a DPU implementation. Additionally, we cover some work on trusted execution environments, which is a different approach to guaranteeing the security of an application running on a cloud server.

3.1 Homomorphic Encryption

3.1.1 Accelerating HE operations using GPUs

The architecture of graphics processing units (GPUs) makes them very well suited for highly parallelizable tasks like HE operations. In addition, GPUs offer much larger memory bandwidth compared to a CPU. Thus, using GPUs is a popular approach to accelerate HE operations [5, 18, 19]. These GPU implementations show $40 - 60\times$ speedups over CPU implementations [5, 18].

However, GPUs are designed for floating point operations and may not handle the high-precision arithmetic (*e.g.*, large integers) required by HE as efficiently as needed. Additionally, while a GPU has a larger memory bandwidth than a CPU, the GPU is still limited by the memory bandwidth [18]. Besides, the GPU memory is often smaller than the main memory accessible by the CPU.

3.1.2 Accelerating HE operations using FPGAs

One popular approach to accelerating HE operations is using field programmable gate arrays (FPGAs) [7, 20]. FPGAs offer great flexibility as they can be reprogrammed to support future developments in HE algorithms. In addition, FPGAs are readily available to the public (*e.g.*, via the AWS cloud) [7]. With FPGAs, Rashmi Agrawal *et al.* [7] achieved $456\times$ and $6.5\times$ better performance than CPU and GPU.

However, FPGAs still face a memory bandwidth bottleneck compared to PIM architectures. Further, HE algorithms are an active area of research and can evolve. Updating an FPGA-based implementation to support new algorithms or optimizations can be more expensive compared to updating software running on a CPU or GPU.

3.2 Processing in-Memory

3.2.1 UPMEM PIM-DRAM

The UPMEM PIM architecture outperforms conventional CPU-centric systems in various real-world applications. Previous studies show that the UPMEM PIM architecture is best suited for applications that are memory-bound on CPU-centric systems [21, 22]. Further, the UPMEM PIM architecture performs best when computing simple operations (*e.g.*, integer addition and subtraction) since the DPUs only have native support for 32-bit integer addition and subtraction [21–23]. With memory-bound applications that mainly perform natively supported operations, the DPUs even manage to outperform a state-of-the-art GPU [23]. However, workloads that are memory-bound but mainly use operations not natively supported by the DPUs become compute-bound on the DPUs very quickly [23]. On such workloads, the DPUs fall behind a CPU and GPU.

Data transfers heavily limit the performance of the UPMEM PIM architecture. Hence, applications that require lots of data transfers between DPUs are less suited for this architecture [21, 22]. Ideally, data should stay in place and be reused as often as possible in order to reach the maximum potential of the UPMEM PIM architecture.

The UPMEM PIM architecture allows the scaling of memory size and bandwidth together with the computing cores. In contrast, adding more CPU cores does not overcome the memory bandwidth scaling issue on CPU-centric systems. This also means the UPMEM PIM architecture is well-suited for highly parallelizable applications. Hence, using more DPUs allows further distribution of the workload, resulting in good performance scaling [23].

3.2.2 Homomorphic Encryption on UPMEM PIM-DRAM

In a similar study to ours, Harshita Gupta *et al.* [24] evaluated using the UPMEM PIM architecture for homomorphic operations. They compared their DPU implementation of the basic homomorphic encryption operations to implementations for the CPU and GPU. Especially for ciphertext additions, the DPUs perform significantly better than a CPU and show similar performance to that of a GPU.

However, when performing integer multiplications, the DPU is slower than the GPU, and the popular HE library SEAL [25] running on the CPU. The lack of native support for 32-bit integer multiplication makes the DPU fall behind the CPU and GPU.

Harshita Gupta *et al.* have implemented their own HE algorithms. In the evaluation they also included results from the popular HE library SEAL, which performed better than their implementation on the CPU. Hence, the question raises if PIM can be used to accelerate a popular HE library like SEAL or HELib.

3.3 Trusted Execution Environment

A completely different approach to guarantee the security of confidential data when running an application on a cloud server is using trusted execution environments (TEEs). TEEs are designed to run alongside the main operating system (OS) but are isolated from it. Consequently, applications running inside the TEE are isolated from the OS and other applications. This ensures the confidentiality of the data being processed inside a TEE.

However, TEEs typically require hardware support, and thus, the security of TEEs depends on the underlying hardware’s root of trust. Additionally, TEEs are vulnerable to certain side-channel attacks [26, 27]. With these security vulnerabilities, data confidentiality cannot be guaranteed. Hence, there is a need for different technologies that can guarantee the security of confidential data.

4

Methodology

We explored three different approaches to which methods we should implement on the DPU. First, we looked at which methods from the HELib library are called the most often. Second, we looked at methods that cause the most cache misses. Third, we implemented parts of the ciphertext addition method, as we know that UPMEM DPUs perform best with integer additions.

We limited ourselves to implementing methods that are not too complex, *i.e.*, are not too long, and do not use advanced C++ features. Firstly, because we must rewrite the code in C since DPU programs have to be written in C. And secondly, the goal is to implement different kinds of methods so that we can evaluate on what kind of workloads the DPUs perform best.

For each method we implemented, we measured the execution times with a varied number of DPUs (32 to 1024, etc). For some methods, we could not use all the way up to 1024 DPUs because we were limited by the implementation or the data size. Additionally, we measured the execution times for different input data sizes for each DPU configuration.

When writing a HELib application, one first sets up a context object defined by some parameters. These parameters define how the HELib objects are represented internally and affect the security and performance of the homomorphic operations. Thus, by changing these context parameters, we can control how large the data is, with which our implemented methods are called.

For simplicity, we evaluated our methods using a small HELib application. The overhead of generating data and calling the methods directly would have been too large. Additionally, since we want to accelerate the HELib library for real-world applications, we only allowed ourselves to change the data size by altering the HELib context parameters.

However, this meant that when we run the program with context parameters that generate large data, all other parts of HELib also have to do a lot more calculations. This increases the overall execution time of the application. Therefore, we did not evaluate with larger data than what is shown in our results since we only had access to the server for a limited amount of time.

We used a server with two Intel Xeon Silver 4210R CPUs for the evaluation. This CPU has 20 cores clocked @2.4 GHz and 13.75 MB of L3 cache. Further, the server is equipped with 2000 DPUs, which we can use for the evaluation.

4.1 Most Called Methods

First, we implemented some of the most frequently called methods for the DPUs. Frequently called methods have a high probability of triggering memory reads and writes, hence data movements between the CPU and memory. We simply added a logging statement at the beginning of each method from HELib to count the number of method calls. Then, we ran a number of different example applications and summed up all the logged method calls.

This way, we found that the most called methods are part of the “Pretty Good FFT” [28] code file used in the HELib library. The PGFFT code file implements various fast fourier transform (FFT) methods that are mainly used for efficient polynomial multiplication.

We implemented the following six methods from the `PGFFT.cpp` file for the DPUs:

- `fwd_butterfly_loop`
- `inv_butterfly_loop`
- `new_fft_layer`
- `new_fft_last_two_layers`
- `new_ifft_layer`
- `new_ifft_first_two_layers`

The `inv_butterfly_loop`, `new_ifft_layer`, and `new_ifft_first_two_layers` methods all perform the inverse operations of the `fwd_butterfly_loop`, `new_fft_layer`, and `new_fft_last_two_layers` methods. This means they perform the same number of additions/subtractions and multiplications on the same data size. Therefore, we only evaluated the performance of the `fwd_butterfly_loop` and the two `new_fft...` methods since we can expect similar results for the other methods. Thus, this report does not discuss these implementations any further.

The `fwd_butterfly_loop`, `new_fft_layer`, and `new_fft_last_two_layers` methods all loop over the input data and perform operations on blocks of four elements from the input arrays. These operations are independent of each other. This meant we could parallelize all three methods very easily to use as many DPUs as possible.

These PGFFT methods do not use any C++ features except from the `complex` class. However, the only methods used from that class are the complex addition, subtraction, and multiplication. This meant that we could easily implement them for the DPUs in C. We just had to define a `struct` that consists of two `double` values and implement the addition, subtraction, and multiplication operations as functions. This also means we must first convert the C++ `complex` input arrays to an array with our C `struct` before writing them to the DPU memories. Subsequently, the resulting array must be converted back to a C++ `complex` array.

For the testing, we used different context parameters to change the input data size (*i.e.*, the number of coefficients), as well as different number of DPUs. However, one issue with the `new_fft_layer` and `new_fft_last_two_layers` methods is that the data size these methods work with cannot be changed by altering the HELib context parameters. We could specifically call these methods with larger data for the evaluation. However, since we want to accelerate the HELib library for real-world applications, the overhead of doing it that way would be too large, and these methods would not be called with larger data anyway. Hence, the methods `new_fft_layer` and `new_fft_last_two_layers` are always called with input arrays of size 1024. This meant we could only use up to 128 DPUs when evaluating `new_fft_layer` and 256 DPUs when evaluating `new_fft_last_two_layers`. The different maximum number of DPUs we could use comes from limitations due to the implementation. The `new_fft_layer` method always processes blocks of 8 values together instead of 4, like the other methods do.

4.2 Most Cache Misses

The main reason for using PIM in any application is to eliminate the memory bottleneck of transferring data between the CPU and the memory. Thus, an obvious choice for a method to implement using the DPU is a method that produces many cache misses. That is because, after each cache miss, data has to be copied from the memory to the CPU cache.

We used the Linux *perf* [29] profiling tool to record the number of cache misses using the following command:

```
$ perf record -e cache-misses <application>
```

Table 4.1 and Table 4.2 list all methods from the HELib library which caused more than 1% of the cache misses running any of the example programs. We recorded the cache misses twice and observed consistent results ($\pm 10\%$ relative to the listed values). We also recorded the cache misses on two different systems. The first system has an AMD Ryzen 9 7900 CPU with 64 MiB L3 cache (Table 4.1). The second system has an Intel Core i5-8250U CPU with 6 MiB L3 cache (Table 4.2).

example program		% of cache-misses		
		additions	multiplications	binary arith.
method	helib::DoubleCRT::toPoly	13.12	37.65	21.37
	helib::new_fft_short	1.33	0.76	0.89
	helib::BitReverseCopy	1.26	4.44	-
	helib::PGFFT::apply	1.04	0.45	1.04
	helib::BluesteinFFT	1.04	0.45	3.08

Table 4.1: Cache misses recorded on system with AMD Ryzen 9 7900 (64MiB L3 cache).

On the system with a large CPU cache (Table 4.1), the `helib::DoubleCRT::toPoly` method produces up to one-third of all cache misses. Then, there is also the `helib::BitReverseCopy` method, which causes slightly more cache misses than the remaining methods.

The `helib::DoubleCRT::toPoly` method is a very complex top-level method that would be very difficult to implement on the DPUs. Thus, regarding the scope of our project, it was not feasible to implement it.

example program		% of cache-misses		
		additions	multiply	binary arith.
method	helib::DoubleCRT::toPoly	9.10	9.38	2.11
	helib::Cmodulus::iFFT	2.16	5.44	0.99
	helib::embeddingLargestCoeff	1.81	1.65	0.16
	helib::CKKS_canonicalEmbedding	1.44	0.55	-
	helib::new_fft_short	1.36	1.54	1.08
	helib::Cmodulus::FFT_aux	1.09	5.58	0.57
	helib::DoubleCRT::do_mul	0.19	1.54	1.60
	helib::embeddingLargestCoeff_x2	-	1.34	0.26
	helib::PGFFT::apply	0.84	1.24	2.76
	helib::DoubleCRT::addPrimesAndScale	-	1.08	1.66
	helib::BluesteinFFT	-	-	15.16
	helib::DoubleCRT::operator=	-	0.65	2.09

Table 4.2: Cache misses recorded on system with Intel Core i5-8250U (6MiB L3 cache).

On the system with the smaller CPU cache (Table 4.2), the same method produced the most cache misses. However, there are different methods that also cause a substantial amount of cache misses, which are:

- `helib::Cmodulus::iFFT`
- `helib::Cmodulus::FFTaux`
- `helib::BluesteinFFT`

These methods are all complex and call methods from other libraries. This also makes it unfeasible to implement them regarding the scope of our project.

Finally, we settled on implementing the `helib::BitReverseCopy` method on the DPUs. This is the only method of those listed in Table 4.1 and Table 4.2 that is feasible to implement. The other methods are too complex, or do not work with a large amount of data.

Using the proper context parameters, the `helib::BitReverseCopy` method only calls the `COBRA` method, which is also defined in `helib::Cmodulus`. Thus, we implemented the `COBRA` method for the DPUs.

The `COBRA` method copies entries from a source array to a target array in a very specific manner. It does not perform any computation. Additionally, the way the entries are copied makes it very difficult to use many DPUs since we have to re-assemble the resulting array by almost copying individual entries from the correct DPU to the right place. We managed to code an implementation that allows us to use more DPUs if the size of the input array increases. However, we did not manage to implement scaling with the number of coefficients. This means we have to do multiple iterations of copying the data to the DPUs, running the DPU program, and copying the data back when the data size increases. More specifically, the program does one iteration with the minimal data size for a specific number of DPUs. From there, doubling the number of coefficients results in doubling the number of iterations the program has to do. Overall, it is not a very suitable method to implement on the DPUs. However, we wanted to evaluate different kinds of methods, so we implemented the `COBRA` method nevertheless.

4.3 Polynomial Addition

The key part of any HE library is the addition of two ciphertexts, also called homomorphic addition. Since the DPUs support native 32-bit integer additions, we can expect to have good performance if we perform additions. Additionally, Harshita Gupta *et al.* [24] have already shown that homomorphic addition can be accelerated using DPUs. Therefore, we explored how the homomorphic addition is implemented in HELib and how we can implement it for the DPUs.

In HELib, ciphertexts are stored as polynomials; thus, the homomorphic addition is implemented as an addition of two polynomials. Like before, we can modify the size of the input polynomials by changing the HELib context parameters. This allows us to evaluate the performance of our implementation on different polynomial sizes. Additionally, we can use any number of DPUs with our implementation.

The polynomial addition is almost exclusively performed when the application adds two ciphertexts. Hence, the method is called only a few times and consequently does not appear in the list of the most called methods or the list with the most cache misses.

5

Evaluation

In this chapter, we present the results of our evaluation. For each implemented method we briefly describe what can be observed from the data.

In a first step, we always measured the execution time of the original HELib implementation running on the CPU. We use the CPU execution time as a baseline to compare the DPU times with. For the DPU implementation, we always measured three different times. The time it takes to copy the data to the DPUs, the time it takes to execute the DPU program, and the time it takes to copy the results from the DPUs back to the memory. We will use the term *DPU execution time* for the DPU time and the term *combined DPU execution time* when speaking of the sum of the data copy times and DPU time.

5.1 FFT Butterfly Loop

For the `fft_butterfly_loop` method, we made measurements using 32 to 1024 DPUs and varying the data size from 2^{12} to 2^{18} coefficients. The coefficients are of type `complex`, which on our test system have a size of 2×8 bytes. Thus, we tested with data sizes from 64 KB to 4 MB.

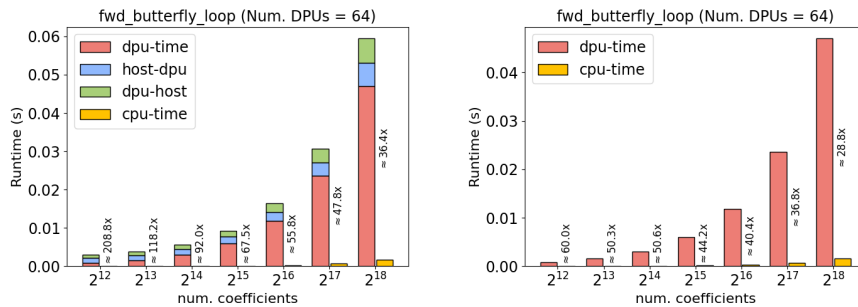
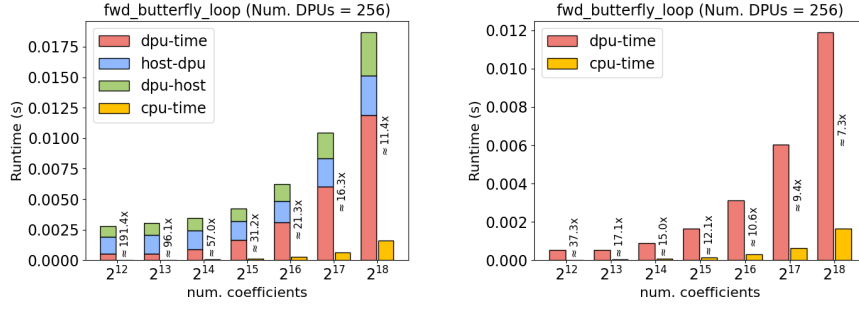
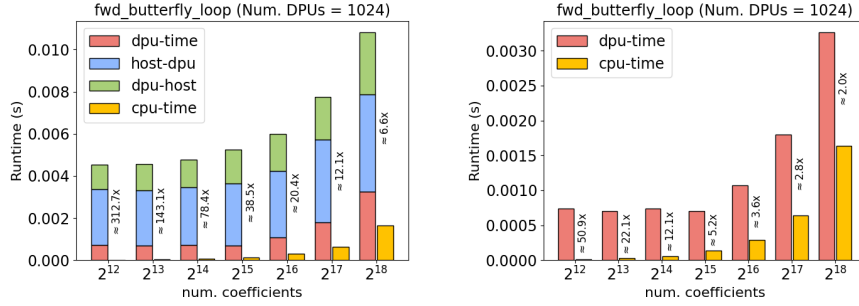


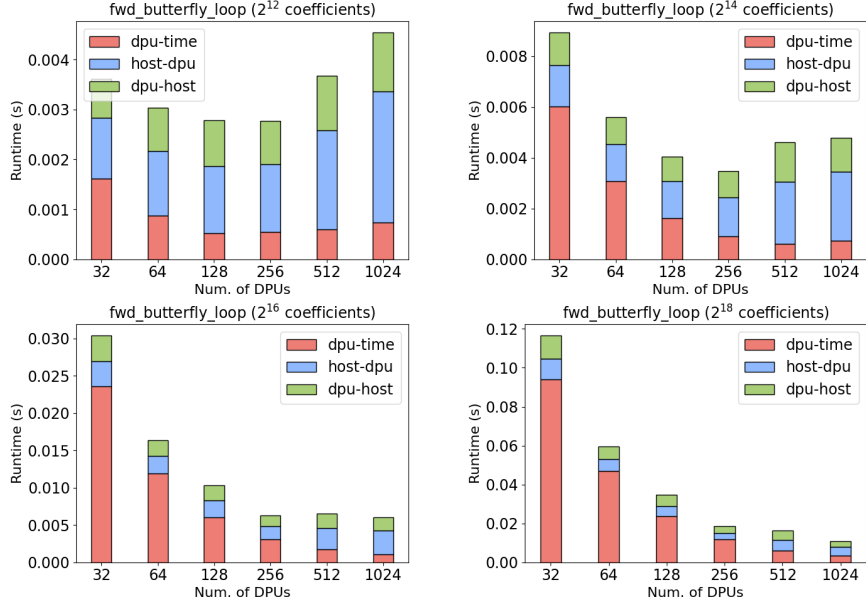
Figure 5.1: `fft_butterfly_loop` method execution time, 64 DPUs vs. single-threaded CPU.

Figure 5.2: `fft.butterfly_loop` method execution time, 256 DPUs vs. single-threaded CPU.Figure 5.3: `fft.butterfly_loop` method execution time, 1024 DPUs vs. single-threaded CPU.

The results for the `fft.butterfly_loop` method show smaller differences between the CPU and DPU execution times when increasing the number of DPUs and data size. However, for the `fft.butterfly_loop`, the DPU execution time is never less than the CPU execution time for all configurations we tested.

With a low number of DPUs, the DPU execution time makes up for the largest part of the combined DPU execution time (Figure 5.1). Using 1024 DPUs, the DPU execution time and the two data copy times each comprise approximately one-third of the combined execution time (Figure 5.3).

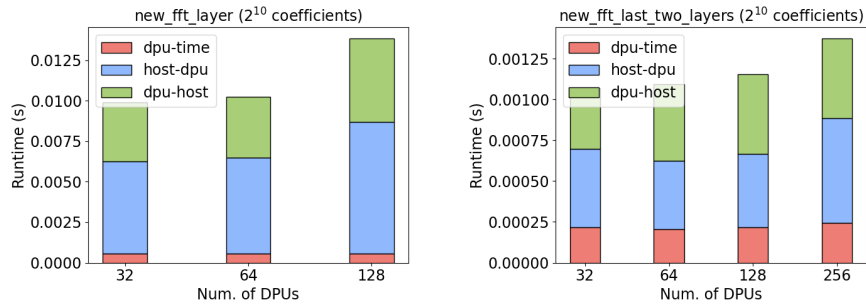
In Figure 5.4, we can observe how the data size affects scaling to more DPUs. For each data size, there is a limit to where increasing the number of DPUs results in a shorter execution time. Noticeably, the data copy times increase from using 256 DPUs to using 512 DPUs, no matter the data size. Hence, the DPU time needs to decrease enough to outweigh the increased data copy times so that the overall execution time can decrease further. For example, with 2^{14} coefficients, we get the best results when using 256 DPUs, while with 2^{18} coefficients, the execution time consistently gets shorter when using more DPUs.

Figure 5.4: `fwd_butterfly_loop` method execution times using different number of DPUs.

5.2 New FFT Layer

For the `new_fft_layer` and `new_fft_last_two_layers` methods, it is not possible to alter the data size by changing the context parameters. Therefore, we only have results for 2^{10} coefficients. These coefficients are of type `complex`, which on our test system have a size of 2×8 bytes. Thus, we tested with a data size of 16 KB.

The results in Figure 5.5 show a slight increase in the combined DPU execution time when using more DPUs. This is the same behaviour as seen with the other methods when the data size is small. Comparing the DPU execution time using 32 DPUs to the CPU execution time, the `new_fft_layer` method DPU execution time is ≈ 980 times longer than the CPU execution time. The combined DPU execution time of the `new_fft_last_two_layers` is ≈ 100 times longer than the CPU execution time.

Figure 5.5: `new_fft_layer` and `new_fft_last_two_layers` methods execution times using different number of DPUs.

5.3 COBRA

The COBRA method simply copies entries from array A to array B in a very specific pattern. This made it very difficult to distribute the task over many DPUs. Further, when calling the method with a large number of coefficients, we have to run multiple iterations of copying the data to the DPUs, running the DPU program, and copying the data back. This is shown in Figure 5.6, Figure 5.7, and Figure 5.8, as the (overall) DPU execution time scales linearly with the number of coefficients. Also, for each number of DPUs, the difference between the DPU execution time and CPU execution time is essentially the same regardless of the number of coefficients.

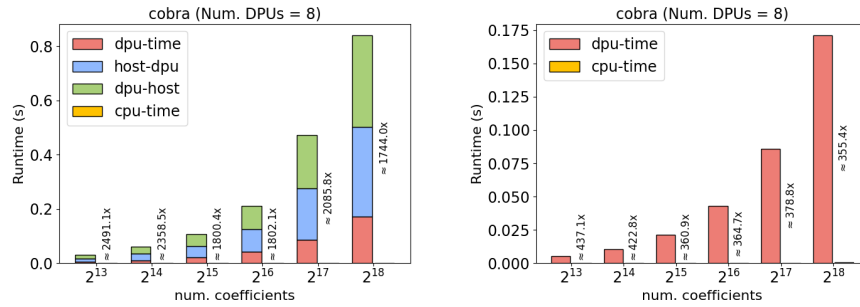


Figure 5.6: COBRA method execution time, 8 DPUs vs. single-threaded CPU.

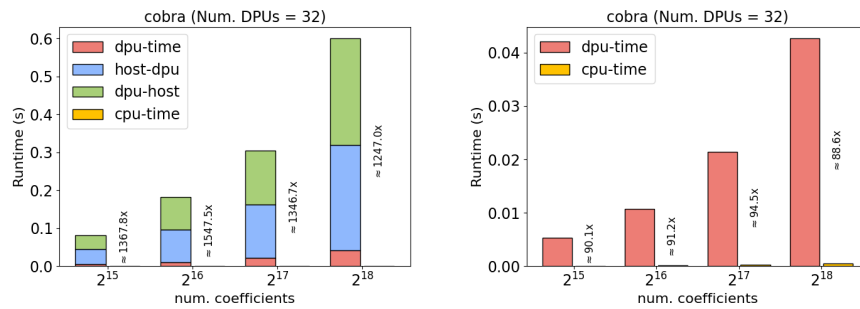


Figure 5.7: COBRA method execution time, 32 DPUs vs. single-threaded CPU.

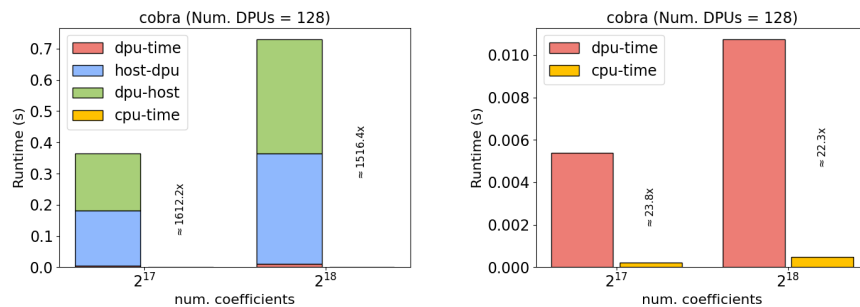


Figure 5.8: COBRA method execution time, 128 DPUs vs. single-threaded CPU.

Using more DPUs on the same number of coefficients mainly decreases the DPU execution time. The data copy times remain mostly the same, as shown in Figure 5.9.

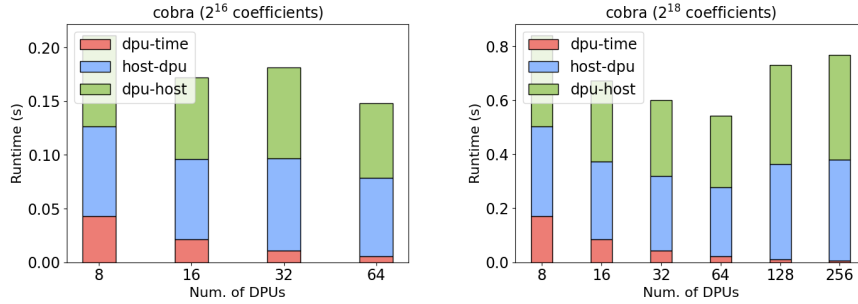


Figure 5.9: COBRA method execution times using different number of DPUs.

5.4 Polynomial Addition

For the polynomial addition method, we made measurements using 32 to 1024 DPUs and varying the polynomial-size from 2^{18} to 2^{24} coefficients. The coefficients are of type `long`, which on our test system have a size of 8 bytes. Thus, we tested with data sizes from 2 MB to 128 MB.

Figure 5.10, Figure 5.11, and Figure 5.12 show the measured execution times using 64, 256, and 1024 DPUs.

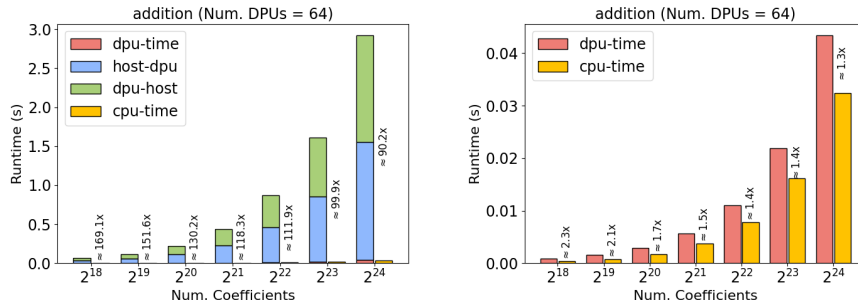


Figure 5.10: Polynomial addition execution time, 64 DPUs vs. single-threaded CPU.

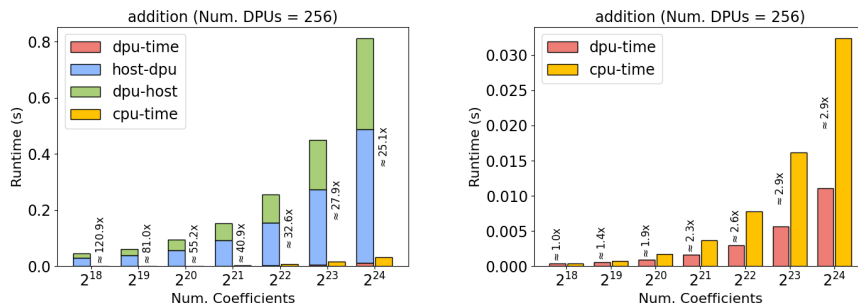


Figure 5.11: Polynomial addition execution time, 256 DPUs vs. single-threaded CPU.

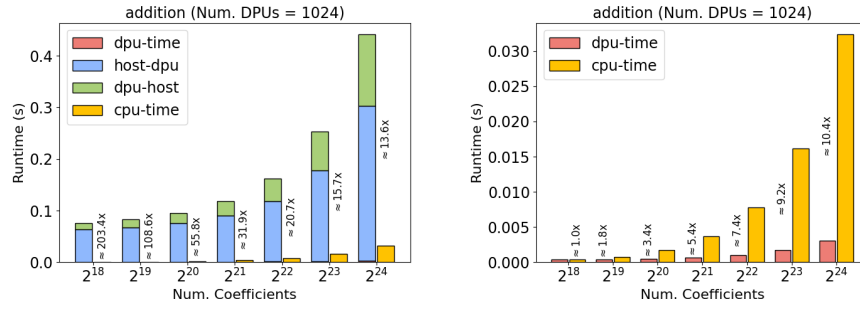


Figure 5.12: Polynomial addition execution time, 1024 DPUs vs. single-threaded CPU.

When using a high number of DPUs (*i.e.*, 128 or more) and when working with a sufficiently large number of coefficients, the DPU execution time is lower compared to the CPU execution time. The ratio between the two execution times grows when using more DPUs and increasing the number of coefficients. With the largest number of DPUs we tested with (*i.e.*, 1024 DPUs), the DPU execution time is ≈ 10 times faster than the CPU execution time.

The results show that the process of copying the data from the CPU to the DPUs, and vice versa, makes up most of the combined DPU execution time. Therefore, when comparing the combined DPU execution time to the CPU execution time, the DPU implementation falls significantly behind. The ratio between these two execution times also shrinks when using more DPUs. Increasing the number of coefficients also decreases that ratio. However, even with the largest number of DPUs and coefficients we tested with (*i.e.*, 1024 DPUs, 2^{24} coefficients), executing the polynomial addition on the CPU is still ≈ 13 times faster than executing on the DPUs.

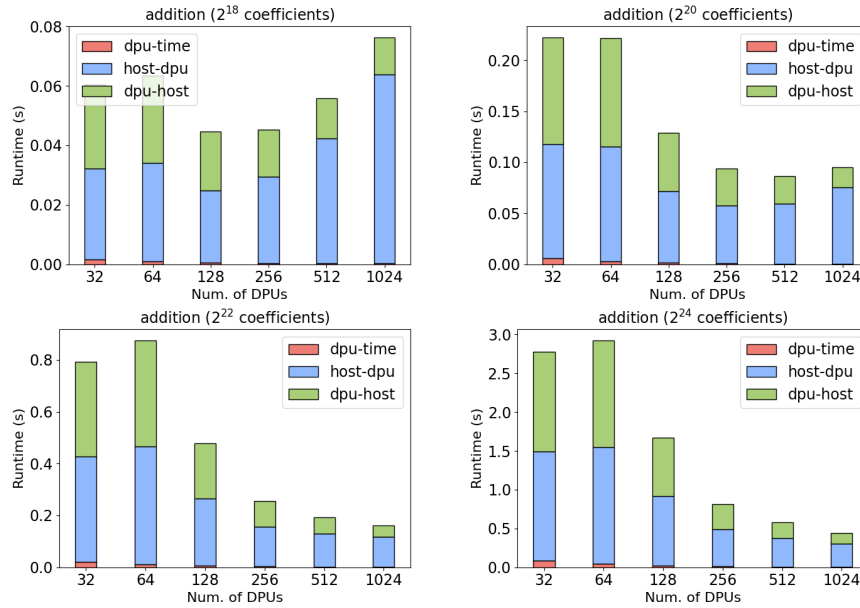


Figure 5.13: Polynomial addition execution times using different number of DPUs.

6

Discussion

From the evaluation of the polynomial addition method, we can state that when the data copy time is not included, the DPUs easily outperform the CPU in terms of computation time. This holds as long as we use at least 256 DPUs. Ideally, one uses 1024 DPUs or even more.

We can also observe that copying the same amount of data to more DPUs takes significantly less time if the data size is large enough (Figure 5.13). The big issue with the polynomial addition method is that the data copy times dominate the combined DPU execution time significantly. The difference between the data copy times and the DPU execution time even increases when using more DPUs. Hence, data transfer times scale less with an increasing number of DPUs than the DPU execution time.

The `fwd.butterfly.loop` method results look more promising than the polynomial addition results because the DPU execution time is approximately equal to both data copy times when using 1024 DPUs (Figure 5.3). Like the polynomial addition method, the data transfer times decrease less than the DPU time when using more DPUs (Figure 5.4).

Key Takeaway 1: To get the best performance from using the UPMEM PIM architecture, data transfers should be minimized. Ideally, data should remain in place and be re-used as often as possible once transferred to the DPU.

We can gain further conclusions by comparing the results and implementations from the polynomial addition and `fwd.butterfly.loop` methods. The polynomial addition method performs one addition and one subtraction with 64-bit integers per coefficient. The `fwd.butterfly.loop` method does one addition, one subtraction, and one multiplication with complex numbers per coefficient. These are three additions, three subtractions, and four multiplications with 64-bit floating-point numbers (Table 6.1).

The results clearly show that the DPUs only support native 32-bit integer additions/subtractions and 8-bit integer multiplications. The DPUs execute the polynomial addition with 2^{18} coefficients in ≈ 0.3 milliseconds (Figure 5.12), while executing the `fwd.butterfly.loop` with 2^{18} coefficients takes ≈ 3 milliseconds (Figure 5.3). The polynomial addition executes ≈ 9 times faster than the `fwd.butterfly.loop` method on the DPUs. On the CPU, the polynomial addition is executed only ≈ 4 times faster (table 6.2).

polynomial addition			fwd.butterfly_loop		
1 ×	addition	64-bit int	3 ×	addition	64-bit float
1 ×	subtraction	64-bit int	3 ×	subtraction	64-bit float
			4 ×	multiplication	64-bit float

Table 6.1: Operations per coefficient, polynomial addition vs.fwd.butterfly_loop method.

	execution time [ms]	CPU	DPU
polynomial addition		0.37	0.38
fwd.butterfly_loop		1.63	3.26
factor ×		4.4	8.6

Table 6.2: Execution times for polynomial addition and fwd.butterfly_loop, CPU vs. 1024 DPUs.

These results show the ideal use case for the DPUs. One needs to perform enough integer additions and subtractions so that the data copying times do not dominate the overall execution time. For the polynomial addition with 2^{24} coefficients and using 1024 DPUs, we would need to do 15 times as many additions and subtractions in order for the DPUs to perform better than the CPU (*i.e.*, 15 additions and 15 subtractions per coefficient).

Key Takeaway 2: The UPMEM DPUs are best suited for applications that only execute simple operations that the DPUs natively support. These are integer addition and integer subtraction.

The `COBRA` method is a perfect example of methods that are not suited at all for the UPMEM PIM architecture. The `COBRA` method simply copies entries from array `A` to array `B` in a very specific pattern. This made it very difficult to distribute the task over many DPUs in the first place. Further, the method does not perform any computations on the data. This just adds the overhead of copying the data to and from the DPUs. Thus, more than the number of cache misses is needed to verify that a method is particularly suited for a DPU implementation. The best result we got for the `COBRA` method is a ≈ 1500 times longer execution time when using the DPUs than when executing on the CPU (Figure 5.8).

The results from the `new_fft_layer` and `new_fft_last_two_layers` methods show what we already observed with the previous evaluations. Using a small number of DPUs and working with little data will never result in any performance gain over the CPU.

6.1 Limitations

We realize it is very difficult to rewrite a complex application that was not designed with PIM in mind for a PIM architecture like UPMEM DPUs. One would need a deeper understanding of the code and algorithms used to code an efficient DPU implementation.

Further, implementing single methods on the DPUs is not a good approach. In order to minimize data movement times, the data should remain in place and be reused as much as possible. Hence, if the results of some calculations are reused in the future, they should be left on the DPU memories. This requires good knowledge of the code and algorithms to know where and when data is reused.

The HELib implementation executed on the CPU always ran on one single CPU core. The methods we implemented on the DPUs can all be parallelized, as we did with our implementation. Considering we ran the tests on a 40-core system, the execution times for the CPU could even be lower if HELib used multiple threads on the CPU.

The Linux *perf* tool records the number of cache misses during the entire program execution. Therefore, two methods might cause the same number of cache misses per call, but if one is called more often, it will be further up in the list. Thus, some methods might cause more cache misses per call than the ones we looked at. We would need to divide the number of cache misses by the number of method calls. This way, we can find methods that are much better suited for a DPU implementation.

7

Conclusion

While we did not succeed in accelerating HELib using the UPMEM PIM-DRAM modules, we did gain some important insights into this technology.

First, we evaluated which parts and methods from the HELib library are best suited to be implemented on the UPMEM DPUs. Previous work has shown that the UPMEM PIM architecture performs best for workloads that perform integer addition/subtraction on large data. Thus, we identified which methods from HELib are either called very frequently, cause many cache misses, do operations on large data, and/or only perform integer addition/subtraction.

We then implemented five different methods, from those identified in the first step, for the UPMEM DPUs. With our implementation, we parallelized each method to use as many DPUs as possible.

Finally, we evaluated the performance of the implemented methods. We measured the execution times using different numbers of DPUs and data sizes. We compared the execution times using the DPUs to the original HELib implementation running on the CPU.

From our evaluation we and gained the following key insights.

Key Takeaway 1: To get the best performance from using the UPMEM PIM architecture, data transfers should be minimized. Ideally, data should remain in place and be re-used as often as possible once transferred to the DPU.

Key Takeaway 2: The UPMEM DPUs are best suited for applications that only execute simple operations that the DPUs natively support. These are integer addition and integer subtraction.

8

Future Work

We suppose designing and implementing a FHE library for the UPMEM PIM hardware from scratch is better than implementing DPU programs into an existing library. While this requires considerable effort, it should be easier to implement reusing of data to minimize data movement in the system. Further, we can first evaluate components of an HE scheme (*e.g.*, polynomial addition/multiplication, ...) in an encapsulated environment. Only executing the implemented method instead of a whole HE application like we did. That way, we can evaluate a component more quickly and determine if it is suited for a DPU implementation. We could easily run tests with larger data, and it would be much more efficient than running a complete HE application every time to evaluate only one single method.

Generally, any future work implementing DPU programs should focus on minimizing data transfers by reusing data. Hence, we should investigate how much reusing data actually affects the performance of DPU programs.

Bibliography

- [1] Peter Mell and Timothy Grance. The nist definition of cloud computing. 2011.
- [2] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *Symposium on the Theory of Computing*, 2009.
- [3] Craig Gentry. Computing arbitrary functions of encrypted data. *Commun. ACM*, 53:97–105, 2010.
- [4] Eyad Saleh, Ahmad AlSa’deh, Ahmad Kayed, and Christoph Meinel. Processing over encrypted data: Between theory and practice. *SIGMOD Rec.*, 45:5–16, 2016.
- [5] Sangpyo Kim, Wonkyung Jung, Jai Hyun Park, and Jung Ho Ahn. Accelerating number theoretic transformations for bootstrappable homomorphic encryption on gpus. *2020 IEEE International Symposium on Workload Characterization (IISWC)*, pages 264–275, 2020.
- [6] Onur Mutlu, Saugata Ghose, Juan G’omez-Luna, and Rachata Ausavarungnirun. A modern primer on processing in memory. *ArXiv*, abs/2012.03112, 2020.
- [7] Rashmi S. Agrawal, Leo de Castro, Guowei Yang, Chiraag Juvekar, Rabia Tugce Yazicigil, Anantha P. Chandrakasan, Vinod Vaikuntanathan, and Ajay M. Joshi. Fab: An fpga-based accelerator for bootstrappable fully homomorphic encryption. *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 882–895, 2022.
- [8] Charles Gouert and Dimitris Mouris. New insights into fully homomorphic encryption libraries via standardized benchmarks. *Proc. Priv. Enhancing Technol.*, 2023:154–172, 2023.
- [9] HELib. <https://homenc.github.io/HELib>.
- [10] Shai Halevi and Victor Shoup. Algorithms in helib. *IACR Cryptol. ePrint Arch.*, 2014:106, 2014.
- [11] Shai Halevi and Victor Shoup. Bootstrapping for helib. *Journal of Cryptology*, 34, 2015.
- [12] Shai Halevi and Victor Shoup. Design and implementation of helib: a homomorphic encryption library. *IACR Cryptol. ePrint Arch.*, 2020:1481, 2020.
- [13] Syed Minhaj Hassan, Sudhakar Yalamanchili, and S. Mukhopadhyay. Near data processing: Impact and optimization of 3d memory system architecture on the uncore. *Proceedings of the 2015 International Symposium on Memory Systems*, 2015.
- [14] Vivek Seshadri, Kevin Hsieh, Amirali Boroumand, Donghyuk Lee, Michael A. Kozuch, Onur Mutlu, Phillip B. Gibbons, and Todd C. Mowry. Fast bulk bitwise and and or in dram. *IEEE Computer Architecture Letters*, 14:127–131, 2015.
- [15] Steve Rhyner, Haocong Luo, Juan G’omez-Luna, Mohammad Sadrosadati, Jiawei Jiang, Ataberk Olgun, Harshita Gupta, Ce Zhang, and Onur Mutlu. Analysis of distributed optimization algorithms on a real processing-in-memory system. *ArXiv*, abs/2404.07164, 2024.

- [16] UPMEM Processing In-Memory (PIM) - UPMEM PIM Tech paper.
- [17] UPMEM SDK Documentation. <https://sdk.upmem.com/2024.1.0/index.html>.
- [18] Wonkyung Jung, Sangpyo Kim, Jung Ho Ahn, Jung Hee Cheon, and Younho Lee. Over 100x faster bootstrapping in fully homomorphic encryption through memory-centric optimization with gpus. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2021:114–148, 2021.
- [19] Ali Şah Özcan, Can Ayduman, Enes Recep Türkoglu, and ErKay Savaş. Homomorphic encryption on gpu. *IEEE Access*, 11:84168–84186, 2023.
- [20] M. Sadegh Riazi, Kim Laine, Blake Pelton, and Wei Dai. Heax: An architecture for computing on encrypted data. *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2019.
- [21] Juan G’omez-Luna, Izzat El Hajj, Ivan Fernandez, Christina Giannoula, Geraldo F. Oliveira, and Onur Mutlu. Benchmarking memory-centric computing systems: Analysis of real processing-in-memory hardware. *2021 12th International Green and Sustainable Computing Conference (IGSC)*, pages 1–7, 2021.
- [22] Joel Nider, Craig Mustard, Andrada Zoltan, John Ramsden, Larry Liu, Jacob Grossbard, Mohammad Dashti, Romaric Jodin, Alexandre Ghiti, Jordi Chauzi, and Alexandra Fedorova. A case study of processing-in-memory in off-the-shelf systems. In *USENIX Annual Technical Conference*, 2021.
- [23] Juan G’omez-Luna, Yu-Yin Guo, Sylvan Brocard, Julien Legriel, Remy Cimadomo, Geraldo F. Oliveira, Gagandeep Singh, and Onur Mutlu. An experimental evaluation of machine learning training on a real processing-in-memory system. *ArXiv*, abs/2207.07886, 2022.
- [24] Harshita Gupta, Mayank Kabra, Juan G’omez-Luna, Konstantinos Kanellopoulos, and Onur Mutlu. Evaluating homomorphic operations on a real-world processing-in-memory system. *2023 IEEE International Symposium on Workload Characterization (IISWC)*, pages 211–215, 2023.
- [25] Microsoft SEAL. <https://www.microsoft.com/en-us/research/project/microsoft-seal>.
- [26] Ben Lapid and Avishai Wool. Cache-attacks on the arm trustzone implementations of aes-256 and aes-256-gcm via gpu-based analysis. *IACR Cryptol. ePrint Arch.*, 2018:621, 2018.
- [27] Guoxing Chen, Sanchuan Chen, Yuan Xiao, Yinqian Zhang, Zhiqiang Lin, and Ten-Hwang Lai. Sgxpectre: Stealing intel secrets from sgx enclaves via speculative execution. *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 142–157, 2018.
- [28] Pretty good FFT (PGFFT). <https://www.shoup.net/PGFFT>.
- [29] perf. https://perf.wiki.kernel.org/index.php/Main_Page.